INSTITUTO TECNOLOGICO DE CULIACAN

INGENIERIA EN SISTEMAS COMPUTACIONALES

PROCESAMIENTO DE ARCHIVO TXT

CIENCIA DE DATOS

DENZEL ARTURO GUTIERREZ CHAVEZ

JUAN CARLOS QUIÑONEZ MADRID

CULIACAN, SINALOA 30/03/26

Link repositorio: https://github.com/CarlosMadrid11/Ciencia-de-datos

En este documento se presenta nuestro proyecto de la materia de Ciencia de Datos, cuyo propósito es el procesamiento de archivos TXT, la limpieza de datos y el análisis exploratorio de datos (EDA). El proyecto se desarrolla en cinco etapas, las cuales se describen a continuación.

ETAPA 1: Carga del archivo TXT

- Durante esta etapa identifique que el archivo presentaba una codificación UTF-8, por lo cual a la hora de cargarlo, optamos por contar con una codificación que funcionara para nuestro avance el cual es ISO-8859-1.
- Utilizamos un parámetro que manejara espacios múltiples, que mantuviera ordenado nuestros datos dentro del archivo
- Por otro lado, fue necesario omitir las primeras 18 filas del documento, ya que contenían información de encabezado que no era relevante para el análisis; esto se logró mediante el uso de skiprows.
- Finalmente, establecimos que la palabra "Nulo" fuera reconocida automáticamente como un valor faltante (NaN), empleando el parámetro na_values, lo que facilitó el posterior proceso de limpieza y análisis de los datos.

```
#En esta parte nos dedicamos a cargar el archivo usando la libreria de Pandas.
import pandas as pd
#Descubrimos que la funcion pd.read_csv tambien sirve para leer datos txt de manera eficiente. En este caso, desde mi computador pongo el
#path completo de donde tengo guardado mi proyecto.
df = pd.read_csv("C:\\CienciaDeDatos\\Modulo 2 - Pandas\\Datasets\\25001.txt", encoding='ISO-8859-1', sep ="\s+",na_values="Nulo",skiprows= 18
```

(carga del archivo)

Etapa 2 — Inspección y corrección de estructura

Durante el procesamiento de los datos, detectamos que los nombres de las columnas no eran correctos debido a la presencia de un doble encabezado en el archivo. Para solucionar este problema, procedimos a renombrar las columnas de manera adecuada, asignándoles los nombres FECHA, PRECIP, EVAP, TMAX y TMIN, lo que permitió una mejor organización y comprensión de la información.

Además, identificamos la existencia de una fila corrupta compuesta por guiones ( ), la cual fue eliminada para evitar inconsistencias en el análisis. Finalmente, realizamos la conversión de la columna FECHA, que originalmente se encontraba en formato de texto, a un tipo de dato datetime64, facilitando así su manipulación y el análisis temporal de los datos.

```
df.axes
✓ 0.0s                                                                    Python

[RangeIndex(start=0, stop=19575, step=1),
 Index(['FECHA', '(MM)', '(MM).1', '(°C)', '(°C).1'], dtype='str')]

#En esta parte cambiamos el nombre de las columnas para tener congruencia en la limpieza de los datos y saber que se esta quitando, ademas de
#ordenar el dataframe base a la fecha.
df.rename(mapper={"(MM)":"PRECIP","(MM).1": "EVAP", "(°C)": "TMAX", "(°C).1": "TIMN"},axis=1, inplace = True)

✓ 0.0s                                                                    Python
```

```python
#Durante esta parte, eliminamos la celda 19574 ya que no proporciona ningun uso (es una serie de lineas divisoras del cuadro ---------)
df.tail()
df = df.drop(19574)
```

✓ 0.0s Python

```python
#Como se analizo en gracias al metodo de dtypes, FECHA queda como str, sin embargo no nos sirve para hacer analisis dentro de la variable
#para ello lo convertimos en un datetime.
df['FECHA'] = pd.to_datetime(df['FECHA'], dayfirst= True)

#En este momento nos arrojaba un error porque leia las lineas ------  de la ultima fila, para eliminarlas se hizo lo siguiente
#df = df.drop(19574)
df.dtypes
```

✓ 0.0s Python

```
FECHA     datetime64[us]
PRECIP           float64
EVAP             float64
TMAX             float64
TIMN             float64
dtype: object
```

Etapa 3 – Diagnostico

Durante esta etapa del análisis, revisamos las estadísticas generales del conjunto de datos utilizando la función describe(), lo que nos permitió obtener una visión inicial sobre la distribución y el comportamiento de las variables. A partir de este análisis, identificamos la presencia de valores nulos en distintas columnas: EVAP contaba con 272 valores faltantes, PRECIP con 40, TMAX con 5 y TMIN con 6.

```python
#Conocemos aqui la media de cada columna para luego revisar si concuerda con los campos rellenados
print(df.describe())
#De esta manera conozco si hay nulos en cada una de las columnas, para saber si puedo modificarle cada una con su media
df.isnull().sum()
```

✓ 0.0s

```
                           FECHA        PRECIP          EVAP          TMAX  \
count                      19574  19534.000000  19302.000000  19569.000000
mean   1989-03-01 18:02:01.385511      2.217452      4.882157     34.468808
min          1961-01-01 00:00:00      0.000000      0.000000     18.000000
25%          1974-05-26 06:00:00      0.000000      3.200000     32.500000
50%          1987-11-18 12:00:00      0.000000      4.500000     35.000000
75%          2004-04-09 18:00:00      0.000000      6.500000     37.000000
max          2018-07-31 00:00:00    217.400000     15.600000     45.500000
std                          NaN      9.015782      2.279340      3.430054

               TIMN
count  19568.000000
mean      16.657420
min       -2.500000
25%       11.000000
50%       16.500000
75%       23.000000
max       29.000000
std        6.248605

FECHA       0
PRECIP     40
EVAP      272
TMAX        5
TIMN        6
dtype: int64
```

ETAPA 4 – Limpieza de Datos

Durante la fase de limpieza de datos, creamos una nueva columna denominada *faltantes*, con el objetivo de identificar aquellas fechas que presentaban registros incompletos. Esto nos permitió tener un mayor control y visibilidad sobre la calidad de la información en cada observación.

Posteriormente, procedimos a rellenar los valores nulos de cada columna utilizando la media correspondiente de cada una, asegurando así que los datos se mantuvieran lo más representativos posible sin introducir sesgos significativos. Finalmente, verificamos que el proceso se hubiera realizado correctamente, confirmando que la función isnull().sum() arrojara un valor de cero en todas las columnas, lo que indica la ausencia total de datos faltantes en el conjunto final.

```
#Conteo de registros faltantes por fecha
columnas = ["PRECIP", "EVAP", "TMAX", "TIMN"]
df["faltantes"] = df[columnas].isnull().sum(axis=1)
df[df["faltantes"] > 0][["FECHA", "faltantes"]]

#La respuesta indicara cuantos nulos hay faltantes en cada fecha, dando un total de 312 columnas con registros faltantes
✓ 0.0s
```

	FECHA	faltantes
0	1961-01-01	1
1	1961-01-02	1
2	1961-01-03	1
3	1961-01-04	1
4	1961-01-05	1
...	...	...
18928	2016-07-23	1
18930	2016-07-25	1
19210	2017-06-01	1
19327	2017-09-26	1
19513	2018-06-01	1

```python
df.head(36)
```

✓ 0.0s

	FECHA	PRECIP	EVAP	TMAX	TIMN	faltantes
0	1961-01-01	0.0	4.882157	25.0	13.0	0
1	1961-01-02	0.0	4.882157	27.5	14.0	0
2	1961-01-03	0.0	4.882157	31.0	11.0	0
3	1961-01-04	0.0	4.882157	32.0	13.0	0
4	1961-01-05	0.0	4.882157	29.0	14.0	0
5	1961-01-06	0.0	4.882157	24.0	14.5	0
6	1961-01-07	0.0	4.882157	28.5	10.0	0

```python
#Revisamos que no quede nigun nulo en las columnas
NulosTotales = df.isnull().sum()
NulosTotales
```

[8] ✓ 0.0s

```
FECHA        0
PRECIP       0
EVAP         0
TMAX         0
TIMN         0
faltantes    0
dtype: int64
```

```python
#Revisamos que no hay duplicados
duplicados = df.duplicated().sum()
duplicados
```

[9] ✓ 0.0s

```
np.int64(0)
```

ETAPA 5 – Visualizacion Final

Finalmente, confirmamos que el conteo de registros en todas las columnas fuera consistente, verificando que cada una presentara un total de 19,574 valores. Esto nos aseguró que, tras el proceso de limpieza y tratamiento de datos faltantes, el conjunto de datos quedara completamente uniforme y listo para su análisis.

```
df.describe()
✓ 0.0s
```

	FECHA	PRECIP	EVAP	TMAX	TIMN
count	19574	19574.000000	19574.000000	19574.000000	19574.000000
mean	1989-03-01 18:02:01.385511	2.217452	4.882157	34.468808	16.657420
min	1961-01-01 00:00:00	0.000000	0.000000	18.000000	-2.500000
25%	1974-05-26 06:00:00	0.000000	3.200000	32.500000	11.000000
50%	1987-11-18 12:00:00	0.000000	4.600000	35.000000	16.500000
75%	2004-04-09 18:00:00	0.000000	6.400000	37.000000	23.000000
max	2018-07-31 00:00:00	217.400000	15.600000	45.500000	29.000000
std	NaN	9.006565	2.263447	3.429615	6.247648

Conclusión

Este proyecto nos dejó ver de primera mano lo que implica trabajar con datos reales: nada llega limpio ni listo para usarse. Desde el simple hecho de cargar el archivo hubo que lidiar con problemas de codificación, encabezados mal formados y filas corruptas que, si no se atienden desde el principio, arruinan cualquier análisis posterior.

Lo más valioso del proceso no fueron las gráficas ni los números finales, sino todo lo que se descubrió en el camino: 272 valores faltantes en EVAP, fechas en formato de texto, duplicados que hay que perseguir. Cada uno de esos detalles recuerda que detrás de un conjunto de datos siempre hay decisiones que tomar, y que una buena limpieza es la diferencia entre sacar conclusiones válidas o simplemente estar contando ruido.

Al final, los 19,574 registros quedaron completos y consistentes, lo cual no es un resultado menor. Es la prueba de que el trabajo meticuloso de las etapas previas valió la pena, y una lección práctica de por qué en Ciencia de Datos se dice que el 80% del tiempo se va en preparar los datos.